Lab 02 Summary

MicroBlaze Soft-Core Processor System with AXI UART Lite on the Nexys A7-100T

Vivado 2025.1 | Vitis Unified IDE 2025.1 | Lab date: 29 Oct 2026

Item	Value
Target device	Xilinx Artix-7 **xc7a100tcsg324-1** (Digilent Nexys A7-100T)
Top-level design	design_microblaze_wrapper (IP Integrator block design *design_microblaze*)
System clock	100 MHz (board oscillator on pin E3 → Clocking Wizard → 100 MHz)
Software	Two standalone (bare-metal) applications: **hello_world** and **uart_echo**
Result	Bitstream generated, timing met, both applications ran on the board over UART (COM6, 9600 baud)

1. Objective

The aim of this lab was to build a complete embedded processor system inside the FPGA fabric. A MicroBlaze soft-core CPU was assembled in the Vivado IP Integrator together with local BRAM memory, an AXI UART Lite peripheral, an interrupt controller, a debug module and the clock/reset infrastructure. After implementation, the hardware was exported as an **.xsa** platform file and used in Vitis to write, build and run C programs that communicate with a PC terminal through the board's USB-UART bridge.

2. Hardware Architecture

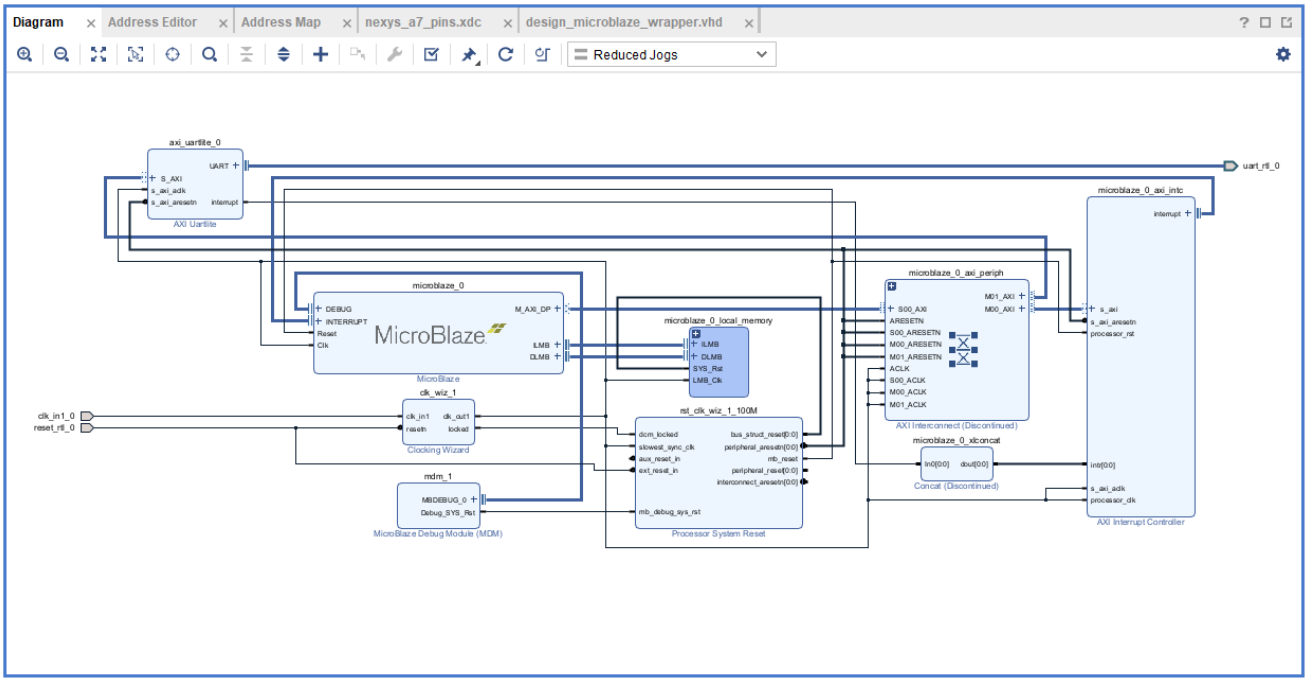


Figure 1 – Vivado IP Integrator block design (design_microblaze)

2.1 IP blocks used

Instance	IP core	Role in the system
microblaze_0	MicroBlaze	32-bit soft RISC CPU, 100 MHz; debug enabled; no caches, no HW multiplier/divider/barrel shifter/FPU (minimal configuration)
microblaze_0_local_memory	LMB BRAM (ILMB + DLMB controllers)	128 KB on-chip Block RAM shared as instruction and data memory, true dual-port
microblaze_0_axi_periph	AXI Interconnect (1 slave → 2 masters)	Connects the CPU M_AXI_DP data port to the UART and the interrupt controller
axi_uartlite_0	AXI UART Lite	Serial link to the PC: 9600 baud, 8 data bits, no parity, 1 stop bit
microblaze_0_axi_intc	AXI Interrupt Controller	Collects peripheral interrupts (fast-interrupt mode) and drives the CPU INTERRUPT input
microblaze_0_xlconcat	Concat	Packs the UART interrupt line into the intr[0:0] vector
clk_wiz_1	Clocking Wizard (MMCM)	Generates the 100 MHz system clock from the board oscillator; active-low reset
rst_clk_wiz_1_100M	Processor System Reset	Produces synchronised resets for CPU, bus, interconnect and peripherals
mdm_1	MicroBlaze Debug Module	JTAG debug access used by Vitis to download and run programs

2.2 Address map (MicroBlaze view)

Segment	Base address	Range	Bus
Instruction memory (ilmb_bram_if_cntlr)	0x0000_0000	128 KB	ILMB
Data memory (dlmb_bram_if_cntlr)	0x0000_0000	128 KB	DLMB
AXI UART Lite registers	0x4060_0000	64 KB	AXI (M_AXI_DP)
AXI Interrupt Controller registers	0x4120_0000	64 KB	AXI (M_AXI_DP)

2.3 Pin constraints (nexys_a7_pins.xdc)

Port	Pin	I/O standard	Function
clk_in1_0	E3	LVCMOS33	100 MHz oscillator (create_clock, period 10.000 ns)
reset_rtl_0	C12	LVCMOS33	CPU_RESET push-button (active low)
uart_rtl_0_rxd	C4	LVCMOS33	UART RX – data from PC
uart_rtl_0_txd	D4	LVCMOS33	UART TX – data to PC

3. Design Flow

- Created the Vivado project for the xc7a100tcsg324-1 part and a new block design.
- Added MicroBlaze and ran **Block Automation** (128 KB local memory, debug module, interrupt controller, clock/reset).
- Added AXI UART Lite (9600 baud) and ran **Connection Automation** to attach it to the AXI interconnect; connected its interrupt through Concat to the interrupt controller.
- Made the clock, reset and UART ports external, validated the design and generated the HDL wrapper.
- Added the Nexys A7 pin constraints file, then ran Synthesis → Implementation → Generate Bitstream.
- Exported the hardware (**design_microblaze_wrapper.xsa**, including bitstream) to Vitis.

- In Vitis: created a standalone platform for microblaze_0, then the hello_world and uart_echo applications; built, programmed the FPGA and ran them while monitoring COM6 at 9600 baud.

4. Software Applications

4.1 Hello World

The template Hello World application was built and run. The serial console printed the expected messages, confirming that the CPU, memory, UART and clock/reset were all working. The disassembly view was also used to inspect the C runtime start-up code (`_crtinit`, `__init`) that calls `main` at address 0x250.

```
Opened with baud rate: 9600
Hello ELE Students
Successfully ran Hello World application
```

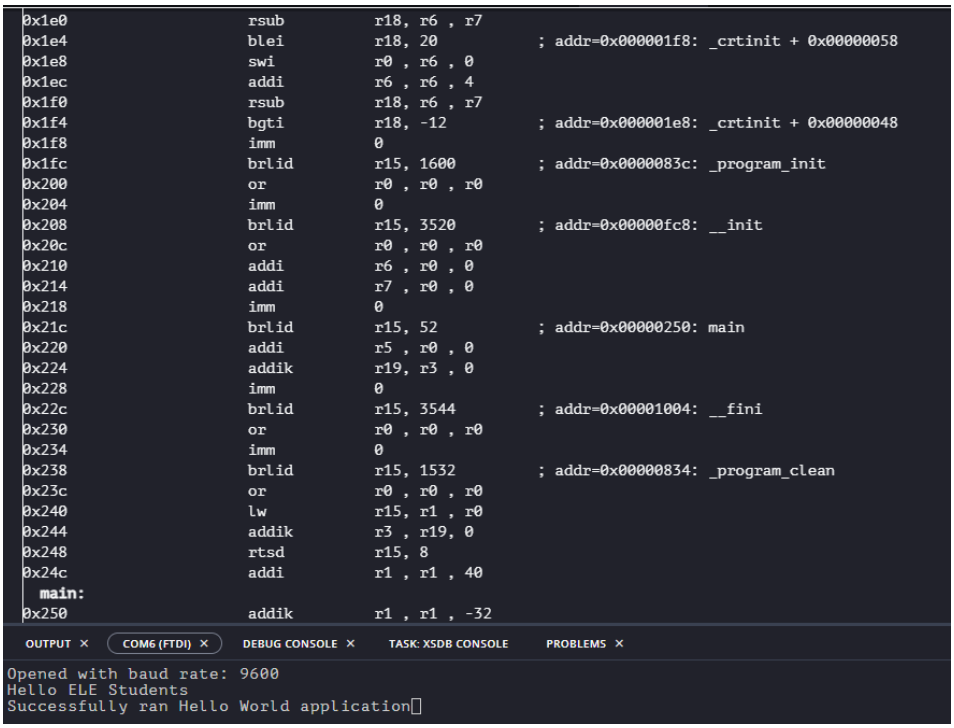


Figure 2 – Vitis: disassembly of the start-up code and Hello World output on COM6

4.2 UART Echo

A second application reads one character at a time from the UART with the blocking `inbyte()` function and prints it back with `xil_printf()`, demonstrating two-way serial communication.

```
#include "xil_printf.h"

int main(void)
{
    xil_printf("Echo test: type a character\r\n");

    while (1)
    {
        char c = inbyte();
        xil_printf("You typed: %c\r\n", c);
    }

    return 0;
}
```

Console output observed (characters **A**, **Enter** and **a** were typed):

```
Echo test: type a character
AYou typed: A
You typed:
aYou typed: a
```

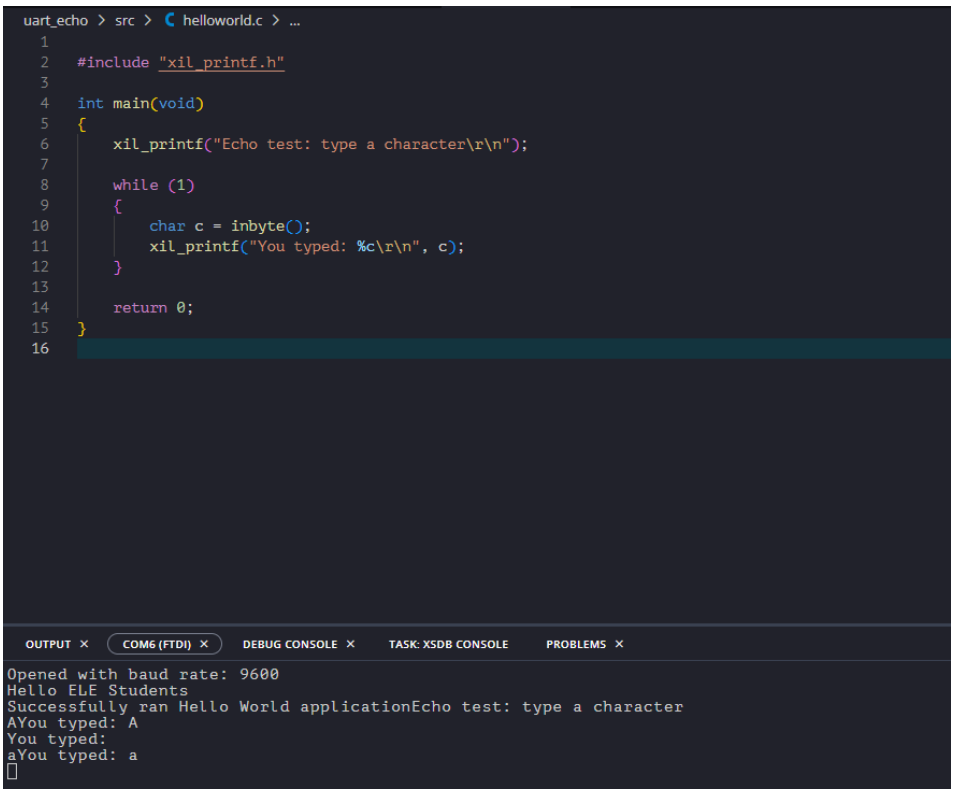


Figure 3 – Vitis: uart_echo source code and terminal output on COM6

5. Implementation Results

5.1 Timing

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	1.803 ns	Worst Hold Slack (WHS):	0.026 ns	Worst Pulse Width Slack (WPWS):	3.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	5892	Total Number of Endpoints:	5892	Total Number of Endpoints:	1920

All user specified timing constraints are met.

Figure 4 – Design Timing Summary (implemented design)

Metric	Value	Comment
Worst Negative Slack (WNS)	1.803 ns	Positive → setup met at 100 MHz (10 ns period)
Worst Hold Slack (WHS)	0.026 ns	Positive → hold met
Worst Pulse Width Slack	3.000 ns	Met
Failing endpoints	0 / 5892	All user-specified timing constraints are met
Estimated max. clock	≈ 122 MHz	1 / (10 ns – 1.803 ns), rough estimate only

Note: the saved impl_1 timing report in the project folder shows WNS = 1.956 ns and WHS = 0.047 ns from a separate implementation run; both runs meet timing.

5.2 Resource utilisation (post-route)

Resource	Used	Available	Utilisation
Slice LUTs	1,515	63,400	2.39 %
LUT as logic	1,345	63,400	2.12 %
LUT as memory	170	19,000	0.89 %
Slice registers (flip-flops)	1,538	126,800	1.21 %
Slices	571	15,850	3.60 %
F7 muxes	109	31,700	0.34 %
Block RAM tiles (RAMB36)	32	135	23.70 %
DSP slices	0	240	0.00 %
Bonded IOBs	5	210	2.38 %
BUFGCTRL (global clock buffers)	4	32	12.50 %
MMCME2_ADV	1	6	16.67 %
BSCANE2 (JTAG debug)	1	4	25.00 %

The processor system is small: under 3 % of the logic is used. Block RAM is the largest consumer (23.7 %), because 32 RAMB36 tiles × 4 KB = 128 KB of local memory was allocated for code and data.

5.3 Power estimate

Component	Power (W)
MMCM (Clocking Wizard)	0.106
Clocks	0.009
Signals	0.007
Block RAM	0.005
Slice logic	0.003
Dynamic total	**0.131**
Device static	0.099
Total on-chip power	**0.230**

Junction temperature is estimated at 26.0 °C (confidence level: medium). The MMCM accounts for about 80 % of the dynamic power.

6. Observations

- **Local echo of typed characters:** in the echo test each typed character appears once before “You typed:” (e.g. `AYou typed: A`). The first character is the terminal’s local echo; the second is the character sent back by the MicroBlaze.
- **Enter key:** pressing Enter sends a carriage return, so the program prints “You typed:” followed by an invisible character.
- **Missing newline:** “Successfully ran Hello World application” and “Echo test…” appear on the same line because the Hello World message does not end with \r\n.
- **Blocking input:** `inbyte()` polls the UART status register and waits; the interrupt controller is present in hardware but was not used by these programs.

7. Conclusion

A working MicroBlaze-based embedded system was designed in Vivado, implemented on the Nexys A7-100T with all timing constraints met at 100 MHz, and programmed from Vitis. Both test programs ran correctly: Hello World verified the basic system, and UART Echo verified two-way serial communication with the PC. The design uses only a small fraction of the FPGA (about 2.4 % of LUTs and 23.7 % of BRAM), which leaves plenty of room for extra peripherals such as GPIO for LEDs and switches, timers, or interrupt-driven UART handling in later labs.

8. Project Files

File	Description
MicroBlaze_Lab02.xpr	Vivado project file
MicroBlaze_Lab02.srcs/sources_1/bd/design_microblaze/design_microblaze.bd	Block design
MicroBlaze_Lab02.srcs/constrs_1/new/nexys_a7_pins.xdc	Pin and clock constraints
MicroBlaze_Lab02.runs/impl_1/design_microblaze_wrapper.bit	FPGA bitstream
design_microblaze_wrapper.xsa	Exported hardware platform for Vitis
utilization_report.txt	Post-route utilisation report
block_design.png, Report_timing_Summary.png, microblaze.png, Screenshot …125115.png	Screenshots used in this summary